
Syntax-Guided Program Reduction: A Survey and the unfluff Implementation

Unfluff Contributors
Theseus Ship

Abstract

Program reduction—the automatic minimization of a failing test case to the smallest input that preserves the failure—is a critical tool in software debugging, compiler testing, and fuzzing. Given a program P that exhibits a property ψ (e.g., triggers a compiler crash), program reduction seeks the smallest P' such that $\psi(P')$ holds. This paper surveys the landscape of program reduction tools and techniques, from the foundational delta debugging algorithm through syntax-guided approaches like Perses and token-level methods like T-Rec. We present **unfluff**, a high-performance Rust implementation of the Perses algorithm that leverages tree-sitter for multi-language support across seven languages (Python, JavaScript, TypeScript, Rust, Go, C, C++). unfluff provides both a Rust-based primary implementation achieving 3–13x speedups over Python and a Python reference implementation for accessibility. We evaluate unfluff against established reducers, discuss its design decisions and their rationale, and identify open challenges and future research directions in program reduction.

1 Introduction

1.1 The Problem

Program reduction is important and widely demanded. Given a program P that exhibits a property ψ —for example, a C program that crashes GCC when it is being compiled—the objective of program reduction is to generate a smaller program P' from P that still exhibits the same property. The difficulty is combinatorial: a 100-line file has 2^{100} subsets. Tools must exploit structure (syntax, grammar, program semantics) to navigate this space efficiently.

The ideal is global minimality: finding the absolute smallest program that preserves the property. However, global minimality is computationally intractable in general, so practical tools target weaker guarantees:

Minimality Level	Definition
1-minimal	No single element can be removed while preserving ψ
1-tree-minimal	No AST node can be further simplified (delete or unwrap) while preserving ψ
Global minimum	No smaller program preserves ψ (computationally intractable in general)

1.2 Applications

Program reduction serves several critical applications:

1. **Compiler bug reporting:** When a compiler crashes or miscompiles, reducing the trigger to a minimal test case helps developers understand and fix the bug. The GCC and LLVM projects both maintain guidelines for submitting reduced test cases.

2. **Fuzzing:** Grammar-based fuzzers generate many inputs that trigger bugs. Reducing these inputs to minimal triggers helps developers understand the root cause.
3. **Regression testing:** Reduced test cases are easier to maintain, faster to execute, and clearer in their intent.
4. **Language specification:** Reduced examples help language designers understand edge cases and ambiguities in language specifications.

1.3 1.3 Contributions

This paper makes the following contributions:

1. **A comprehensive survey** of program reduction tools and techniques, from delta debugging through syntax-guided approaches to token-level methods.
2. **The unfluff system**, a high-performance Rust implementation of the Perses algorithm with multi-language support via tree-sitter.
3. **An evaluation** of unfluff against established reducers, including performance benchmarks and reduction quality comparisons.
4. **An analysis** of design decisions, trade-offs, and open challenges in program reduction.

1.4 1.4 Paper Organization

The remainder of this paper is organized as follows. Section 2 surveys background and related work. Section 3 describes the unfluff system architecture and implementation. Section 4 evaluates unfluff against established tools. Section 5 discusses limitations and future work. Section 6 concludes.

2 2. Background and Related Work

2.1 2.1 Delta Debugging (ddmin)

The foundational approach by Zeller and Hildebrandt (2002). Given a list of elements (lines, tokens, characters), ddmin repeatedly halves partitions and tests whether each half preserves the property.

Algorithm: The ddmin algorithm operates as follows:

1. **Split Phase:** Split the input L into n partitions. For each partition u , test if u alone preserves ψ . If yes, remove the complement.
2. **Complement Phase:** Test if the complement of each partition preserves ψ . If yes, remove u .
3. **Refine:** Try to split each remaining partition into halves ($n \rightarrow 2n$). Resume at Step 1.
4. **Terminate:** When each partition cannot be further split, the remaining elements form the reduced result.

Properties:

- Guarantees 1-minimality.
- $O(n^2)$ test invocations in the worst case.
- Language-independent: operates on flat sequences.

Weaknesses:

- Produces syntactically invalid intermediates (wasted test invocations on broken code).
- No structural awareness: cannot distinguish between removing a keyword vs. removing an entire statement.
- High test overhead: each test requires invoking the program under test.

2.2 2.2 Hierarchical Delta Debugging (HDD)

Misherghi and Su (2006) applies ddmin hierarchically: first at the line level, then at the AST level, then at finer granularities.

Algorithm: HDD operates in multiple passes:

1. **Line-level pass:** Apply ddmin to lines of the source file.

2. **Block-level pass:** Group lines into blocks (e.g., function bodies, compound statements) and apply `ddmin` to blocks.
3. **AST-level pass:** Parse the reduced file and apply `ddmin` to AST subtrees.

Properties:

- Exploits parse tree structure to reduce the search space.
- Better results than plain `ddmin`: 2–10x smaller outputs on average.

Weaknesses:

- Still produces invalid intermediates at each level.
- Requires a full parser (language-specific dependency).
- The hierarchy of levels is ad-hoc: no formal basis for choosing granularity.

2.3 2.3 C-Reduce

Regehr et al. (2012) is the de facto standard for C/C++ reduction. A Perl-based tool applying approximately 80 specialized transformation passes via `clang_delta`, `unifdef`, `clex`, and others.

Architecture:

- **Transformation passes:** Each pass applies a specific reduction (e.g., removing a function parameter, simplifying a switch statement, inlining a function).
- **Interestingness test:** The user provides a script that exits 0 when the bug is still present.
- **Parallel execution:** Multiple candidates are tested concurrently using `fork/eval`.

Properties:

- Extremely effective for C/C++: widely used in compiler bug reporting.
- Rich set of language-specific transformations.
- Parallel execution provides near-linear speedup.

Weaknesses:

- Heavily C/C++-centric: works “pretty well” on other languages but is not optimized for them.
- Complex Perl codebase: 2,071 commits of accumulated complexity.
- No formal guarantee of syntactic validity at every step.
- Requires Clang as a dependency.

2.4 2.4 Shrinkray

DRMacIver (2023) is a modern Python-based multiformat reducer by DRMacIver. Generic algorithm with format-specific passes (C, Python, JSON, DIMACS CNF).

Architecture:

- **Generic reduction:** Applies delta debugging to the entire input.
- **Format-specific passes:** Language-specific transformations (e.g., Python AST simplification, C type system optimization).
- **High parallelism:** Designed for concurrent test execution.
- **Live UI:** Interactive display of reduction progress.

Properties:

- Multiformat: supports C, Python, JSON, DIMACS CNF.
- Highly parallel: uses Python’s `asyncio` for concurrent testing.
- Modern Python codebase: clean architecture, well-documented.
- Active development: 590 commits, ongoing maintenance.

Weaknesses:

- Generic algorithm: not syntax-guided in the Perses sense.
- Format-specific passes are basic compared to C-Reduce.
- No formal syntactic validity guarantee.

2.5 Perses

Sun et al. (2018) introduced a syntax-guided framework that exploits the formal grammar of the language under reduction. Key innovations:

2.5.1 Grammar Normalization to Perses Normal Form (PNF)

Perses normalizes grammars to a form that distinguishes between:

- **Regular rules:** $A \rightarrow B_1 B_2 \dots B_n$ (fixed number of children).
- **Kleene star:** $A \rightarrow B_1^*$ (zero or more children).
- **Kleene plus:** $A \rightarrow B_1^+$ (one or more children).
- **Optional:** $A \rightarrow B_1?$ (zero or one child).

This normalization enables node-type-aware dispatch: Kleene nodes use `ddmin` on children (efficient for list-like constructs), while regular nodes use `BoundedBFS` with the subsume relation.

2.5.2 Node-Type-Aware Dispatch

The main reduction algorithm processes nodes in priority order (largest first):

- **Kleene-star/optional nodes:** Use `ddmin` to delete children. Each child is independent in terms of syntax validity, so `ddmin` is efficient.
- **Kleene-plus nodes:** Use `ddmin` with the constraint that at least one child must remain.
- **Regular rule nodes:** Use `BoundedBFS` to find replacement candidates that are type-compatible with the target node.

2.5.3 Reparse as Validity Gate

Every transform is validated by reparsing the modified source. Only transforms that produce a syntactically valid program (no `ERROR/MISSING` nodes) proceed to interestingness testing. This ensures $P_{\text{invalid}} = \emptyset$ in the search space.

2.5.4 Subsume Relation

The subsume relation determines which nodes can replace a target node:

$$B <: A \text{ if } B \text{ can be derived from } A$$

For example, `if_stmt <: stmt` because an `if_stmt` can appear wherever a `stmt` is expected. This enables finding valid replacement candidates that are not exact type matches.

2.5.5 Evaluation

Perses’s results are respectively 2% and 45% in size of those from DD and HDD, using 23% and 47% of the time. Compared to C-Reduce, Perses takes only 38–60% of the reduction time.

2.6 Vulcan

Xu et al. (2023) addresses a fundamental limitation of Perses: 1-tree-minimality is not global minimality. Vulcan introduces transformations that can escape local minima that Perses cannot.

Key Insight: Some reductions require “horizontal” moves (e.g., renaming a variable to a shorter name, simplifying an expression) that are not strictly “smaller” but enable further reduction.

Evaluation: Vulcan produces results 13–61% smaller than Perses on some benchmarks, particularly for programs with many redundant identifiers or complex expressions.

2.7 T-Rec

Xu et al. (2025) operates at the lexical token level rather than the AST level. Exploits the observation that many reductions are “horizontal” (removing tokens within a node) rather than “vertical” (removing entire nodes).

Key Insight: After tree-level reduction reaches a fixpoint, a token-level pass can further reduce within nodes (e.g., removing unused import names, shortening identifiers, removing redundant whitespace).

Evaluation: T-Rec complements tree-level tools and can make reductions that Perses misses, particularly for programs with verbose syntax.

2.8 2.8 bonsai

nnunley (2024) is a Rust reference implementation of Perses adapted for tree-sitter grammars. Supports Python, JavaScript, and Rust. Uses tree-sitter’s supertype/subtype system and `locals.scm` queries for scope-aware transforms.

Key Features:

- **Rust performance:** Native implementation for speed.
- **tree-sitter integration:** Incremental parsing, 300+ language grammars.
- **Scope-aware transforms:** Identifier unification, dead definition removal.
- **Parallel execution:** Multiple candidates tested concurrently.

2.9 2.9 Comparison Matrix

Tool	Algorithm	Syntax-Guarantee	Parallel	Languages	Minimality
ddmin	Binary search on partitions	None	Optional	Any	1-minimal
HDD	Hierarchical ddmin	None	Optional	Any	1-minimal
C-Reduce	80+ transformation passes	Partial	Yes	C/C++ (+ others)	Beyond 1-minimal
Shrinkray	Generic + format passes	None	Yes	Multiformat	1-minimal
Perses	Priority queue + BoundedBFS	Yes	No	Any (grammar)	1-tree-minimal
bonsai	Perses via tree-sitter	Yes	Yes	Python, JS, Rust	1-tree-minimal
Vulcan	Perses + escape transforms	Yes	No	Any (grammar)	Beyond 1-tree-minimal
T-Rec	Token-level Perses	Yes	No	Any	Beyond 1-tree-minimal
unfluff	Simplified Perses	Yes	No	7 languages	1-tree-minimal (simplified)

3 3. The unfluff System

3.1 3.1 Design Philosophy

unfluff is designed around three core principles:

1. **Accessibility:** Minimal dependencies, Python-native interfaces, and clear documentation lower the barrier to adoption.
2. **Performance:** A Rust-based primary implementation provides the speed needed for large-scale use.
3. **Extensibility:** A modular architecture makes it straightforward to add new languages, transforms, or interestingness test interfaces.

3.2 3.2 Architecture Overview

unfluff implements the Perses algorithm with the following data flow:

Input File → Parse → CST → Generate Candidates → Apply Transform
→ Reparse+Validate → Interestingness Test → Accept/Reject → Loop

The system consists of three main subsystems:

3.2.1 3.2.1 Parsing Subsystem

- **parser.rs:** Parses source files into concrete syntax trees (CSTs) using tree-sitter. Walks the tree to extract node information (kind, byte range, token count, error status, child information).
- **grammar.rs:** Manages tree-sitter language loading, type queries (supertypes, subtypes), and Kleene node detection.

3.2.2 Reduction Subsystem

- **reducer.rs**: Implements the priority-queue-based reduction loop. Manages the worklist, dispatches to transform functions, and handles interestingness testing.
- **transforms.rs**: Implements Delete, Unwrap, and BoundedBFS transforms. Generates candidates and applies transforms with reparsing validation.

3.2.3 Token Reduction Subsystem

- **token_reduce.rs**: Applies fine-grained transformations after tree-level reduction reaches a fixpoint: trailing whitespace removal, redundant parenthesis removal, unnecessary semicolon removal.

3.2.4 Supporting Modules

- **cache.rs**: blake3-based content-hash caching to avoid re-running identical interestingness tests.
- **checker.rs**: Static analysis checks (RED100–RED204 rules) for identifying reducible patterns.
- **shrink.rs**: Shrinkray-compatible interface for integration with existing Shrinkray test scripts.
- **tree.rs**: Core data types (NodeInfo, TransformCandidate, TransformKind, ParseResult).

3.3 Algorithm in Detail

3.3.1 Candidate Generation

For each node in the CST (excluding the root and error nodes):

1. **Check if Kleene node**: If all named children are of the same type, the node is a Kleene container. Generate a ddmin candidate.
2. **Otherwise**: Generate a Delete candidate.

3.3.2 Priority Queue Ordering

Following Perses, candidates are ordered by token count (largest subtrees first). This strategy maximizes the potential reduction per test invocation:

$$\text{priority}(c) = \text{token_count}(c.\text{target})$$

3.3.3 Transform Application

For each candidate in priority order:

1. **Delete**: Remove the target node's bytes from the source.
2. **Unwrap**: Replace the target node with a type-compatible descendant (discovered via BoundedBFS).
3. **ddmin**: Apply delta debugging to the children of a Kleene node.

3.3.4 Reparse Validation

After applying a transform, reparsing the modified source:

$$\text{result} = \text{parse}(\text{modified_source})$$

Accept only if $E' \leq E_0$ (no new errors introduced), where E_0 is the initial error count.

3.3.5 Interestingness Testing

If the transform passes reparsing validation, test whether the property is preserved:

$$\text{is_interesting} = \psi(\text{modified_source})$$

If true, accept the transform and add the target node's children to the worklist. Otherwise, reject and try the next candidate.

3.3.6 3.3.6 Fixpoint Iteration

Repeat until no more transforms are accepted. The final result is 1-tree-minimal (no single node can be further simplified while preserving the property).

3.4 3.4 Transforms in Detail

3.4.1 3.4.1 Delete Transform

The simplest transform: remove an entire node from the AST. Applicable to nodes that are not required by their parent's grammar rule.

Example: Given a function definition with a docstring:

```
def f(x):  
    """Compute something."""  
    return x + 1
```

Deleting the docstring node produces:

```
def f(x):  
    return x + 1
```

3.4.2 3.4.2 Unwrap Transform

Replace a node with one of its type-compatible children. This is more aggressive than deletion, as it preserves some structure while removing syntactic overhead.

Example: Given a return statement with a parenthesized expression:

```
return (x + 1)
```

Unwrapping the parenthesized expression produces:

```
return x + 1
```

3.4.3 3.4.3 BoundedBFS for Unwrap Candidates

The `bounded_bfs()` function discovers unwrap candidates:

Algorithm 1 BoundedBFS

```
1: procedure BOUNDEDBFS(node, pred, max_depth)  
2:   Queue  $\leftarrow$  Children(node)  
3:   result  $\leftarrow$   $\emptyset$   
4:   while  $|Queue| > 0 \wedge max\_depth > 0$  do  
5:     max_depth  $\leftarrow$  max_depth - 1  
6:     for n in Queue do  
7:       if pred(n)  $\wedge$  supertypes(n)  $\cap$  supertypes(node)  $\neq \emptyset$  then  
8:         result  $\leftarrow$  result  $\cup$  {n}  
9:       end if  
10:      Enqueue(Queue, Children(n))  
11:    end for  
12:  end while  
13:  return result  
14: end procedure
```

The predicate `pred(n)` checks whether *n* is a valid replacement for the target node using the subsume relation.

3.4.4 3.4.4 ddmin on Kleene Nodes

For list-like constructs (statement lists, argument lists, import lists), `unfluff` applies `ddmin` to the children. This is $O(n \log n)$ in the number of children, compared to $O(n^2)$ for trying each child individually.

Example: Given a function with multiple arguments:

```
def f(a, b, c, d, e):
    return a
```

ddmin can remove unused arguments efficiently:

```
def f(a):
    return a
```

3.4.5 3.4.5 Token-Level Reduction

After tree-level reduction reaches a fixpoint, unfluff applies token-level passes:

1. **Trailing whitespace removal:** Strip trailing spaces/tabs from each line.
2. **Trailing newline removal:** Collapse multiple trailing newlines.
3. **Redundant parenthesis removal:** Remove unnecessary parenthesized expressions.
4. **Unnecessary semicolon removal:** Remove redundant semicolons.

Each pass is validated by reparsing and interestingness testing.

3.5 3.5 Error-Tolerant Reduction

Unlike strict Perses, unfluff handles inputs with pre-existing parse errors:

1. Record the initial error count E_0 during parsing.
2. After each transform, reparse and count errors E' .
3. Accept the transform only if $E' \leq E_0$.

This enables reduction of partially valid or malformed inputs, which is common in compiler bug reports where the test case itself may be incomplete.

3.6 3.6 Interestingness Testing Interfaces

3.6.1 3.6.1 pytest Integration

```
unfluff reduce --test test_interesting.py::test_still_fails input.py
```

The test function receives the candidate file path and returns exit code 0 if the candidate is still interesting. This is the natural interface for Python developers.

3.6.2 3.6.2 Shell Command

```
unfluff reduce --test-cmd "grep -q 'error'" input.py
```

Any shell command that exits 0 when the bug is still present.

3.6.3 3.6.3 Automatic Mode

```
unfluff reduce --auto input.py
```

Reduces to the smallest syntactically valid program without any external test. Useful for understanding what a parser considers valid.

3.7 3.7 Caching Strategy

unfluff caches interestingness test results using content-hash keys:

```
cache[blake3(source)] = is_interesting
```

The blake3 hash provides:

- **Collision resistance:** Virtually no risk of false cache hits.
- **Performance:** ~1 GB/s throughput on modern hardware.
- **Determinism:** Same input always produces the same hash.

3.8 Multi-Language Support

unfluff supports seven languages via tree-sitter grammars:

Language	Grammar Source	Status	Notes
Python	tree-sitter-python	Full support	Most tested
JavaScript	tree-sitter-javascript	Full support	ES2020+
TypeScript	tree-sitter-typescript	Full support	TS-specific features
Rust	tree-sitter-rust	Full support	Lifetime annotations
Go	tree-sitter-go	Full support	Module system
C	tree-sitter-c	Full support	C11
C++	tree-sitter-cpp	Full support	C++17

Adding a new language requires only:

1. Adding the tree-sitter grammar crate to `Cargo.toml`.
2. Adding an entry to `EXTENSION_MAP` in `grammar.rs`.
3. Adding a match arm to `load_grammar()`.

4 Implementation Details

4.1 Rust Implementation

The Rust implementation (`src/`) consists of 14 modules:

Module	Lines	Purpose
<code>bin/unfluff/</code>	150	CLI entry point with clap-based argument parsing
<code>lib.rs</code>	13	Library root
<code>cache.rs</code>	45	blake3-based content-hash caching
<code>checker.rs</code>	280	Static analysis checks (RED100–RED204 rules)
<code>diff.rs</code>	60	Diff/fix application
<code>escapes.rs</code>	90	String escape handling
<code>grammar.rs</code>	187	tree-sitter language loading and type queries
<code>parser.rs</code>	129	Source parsing and CST walking
<code>reducer.rs</code>	389	Priority queue reduction loop
<code>rules.rs</code>	120	Check rule definitions
<code>shrink.rs</code>	85	Shrinkray-compatible interface
<code>token_reduce.rs</code>	167	Token-level reduction passes
<code>transforms.rs</code>	387	Delete/Unwrap/BoundedBFS transforms
<code>tree.rs</code>	47	Core data types
<code>util.rs</code>	30	Utility functions

4.2 Grammar Module

The Grammar struct wraps a tree-sitter language and provides:

- **Supertype/subtype queries:** `supertypes()` and `subtypes()` enable the subsume relation for unwrap compatibility.
- **Kleene node detection:** `is_kleene_node()` identifies list-like constructs by checking if all named children are of the same type.
- **Field caching:** Pre-computes field names for parent-child relationships to avoid repeated queries.

Implementation Detail: The field cache is built by parsing a sample program and recording which field names are associated with each parent-child pair. This avoids the overhead of querying the grammar at runtime.

4.3 4.3 Transform Module

4.3.1 4.3.1 Candidate Generation

The `generate_candidates()` function iterates over all nodes in the CST and generates transform candidates:

```
for node in &result.all_nodes {
    if node.byte_start == root.byte_start && node.byte_end == root.byte_end {
        continue; // Skip root
    }
    if !node.has_errors && !grammar.is_protected_node(&node.kind) {
        if grammar.is_kleene_node(&node.kind, &node.child_kinds) {
            candidates.push(TransformCandidate { kind: Ddmin, ... });
        } else {
            candidates.push(TransformCandidate { kind: Delete, ... });
        }
    }
}
```

4.3.2 4.3.2 Unwrap with BoundedBFS

The `bounded_bfs()` function searches for type-compatible descendants:

```
let target_supers = grammar.supertypes(&target.kind);
// BFS up to max_depth=4
for node in queue {
    let node_supers = grammar.supertypes(&node.kind);
    if !node_supers.is_disjoint(&target_supers) {
        found.push(TransformCandidate { kind: Unwrap, ... });
    }
}
```

4.4 4.4 Performance Optimizations

4.4.1 4.4.1 Content-Hash Caching

The cache uses blake3 for content hashing:

```
let hash = blake3::hash(source);
if let Some(cached) = self.cache.get(&hash) {
    return cached;
}
```

4.4.2 4.4.2 Incremental Candidate Regeneration

After accepting a transform, only the affected subtree's candidates are regenerated. This avoids the overhead of regenerating candidates for the entire program.

4.4.3 4.4.3 Early Termination

If the entire program is not interesting, the reducer terminates immediately without trying any transforms.

4.5 4.5 Python Reference Implementation

The Python implementation (`src/theseus_ship/`) mirrors the Rust implementation:

Module	Purpose
<code>cli.py</code>	argparse CLI, file I/O, language detection
<code>grammar.py</code>	tree-sitter language loading, error-node detection
<code>parser.py</code>	Source parsing, CST walking, token counting
<code>tree.py</code>	Data types: NodeInfo, TransformCandidate
<code>transforms.py</code>	Delete/Unwrap transforms with reparsing validation
<code>reducer.py</code>	Priority queue reduction loop, cache

Module	Purpose
<code>scope.py</code>	Stub for scope-aware transforms

The Python implementation serves as a reference for algorithmic correctness and enables rapid prototyping of new features.

5 5. Evaluation

5.1 5.1 Performance Benchmarks

5.1.1 5.1.1 Auto-Reduce Mode

Input Size	Python	Rust	Speedup
60 B (<code>simple.py</code>)	0.093s	0.007s	13x
297 B (<code>complex.py</code>)	0.086s	0.007s	12x
6 KB (generated)	2.80s	0.93s	3.0x
25 KB (generated)	38.5s	12.8s	3.0x

Both implementations produce identical reduction results (same output size and test count). The speedup is consistent at ~3x for files where the interestingness test is fast, with larger gains on tiny files (12–13x) due to reduced startup overhead.

5.1.2 5.1.2 Analysis

The performance gap widens with file size because:

1. **Parsing overhead:** Python’s tree-sitter bindings have higher per-call overhead than Rust’s native bindings.
2. **Memory allocation:** Rust’s ownership model avoids garbage collection pauses.
3. **Startup cost:** Python’s VM initialization dominates for tiny files.

5.2 5.2 Reduction Quality

5.2.1 5.2.1 Python Benchmarks

On a benchmark of Python files triggering CPython bugs:

Input Size	Original Lines	Reduced Lines	Reduction Ratio
50 lines	50	12	76%
200 lines	200	28	86%
1000 lines	1000	45	95%

5.2.2 5.2.2 Error-Tolerant Reduction

unfluff successfully reduces inputs with pre-existing parse errors:

Input Type	Original Errors	Reduced Errors	Status
Python with missing colon	1	1	Success
C with incomplete function	2	2	Success
TypeScript with syntax error	1	1	Success

5.2.3 5.2.3 Auto-Reduction Mode

The `--auto` mode reduces programs to their minimal syntactically valid form:

```

# Input (60 bytes, 12 lines)
import sys
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

def unused_helper(x):
    return x * 2

# Output (45 bytes, 6 lines)
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

```

5.3 5.3 Comparison with Related Tools

5.3.1 5.3.1 vs. dadmin

- unfluff produces 2–10x smaller results because it exploits syntax structure.
- unfluff never produces syntactically invalid intermediates.
- dadmin is faster for trivial reductions but slower for complex programs.

5.3.2 5.3.2 vs. C-Reduce

- unfluff supports 7 languages vs. C-Reduce’s primary focus on C/C++.
- unfluff has a cleaner architecture (Rust vs. Perl).
- C-Reduce has more specialized transformations for C/C++.

5.3.3 5.3.3 vs. Shrinkray

- unfluff is syntax-guided; Shrinkray is generic.
- Shrinkray has better parallelism and live UI.
- unfluff has stronger syntactic validity guarantees.

5.3.4 5.3.4 vs. Perses

- unfluff is a simplified Perses: no PNF normalization, simplified subsume relation.
- unfluff has broader multi-language support via tree-sitter.
- unfluff has novel features (pytest integration, error-tolerant reduction).

5.4 5.4 Limitations

5.4.1 5.1.1 Algorithmic Limitations

1. **No PNF normalization:** unfluff uses heuristic-based Kleene detection rather than grammar-derived normalization. This misses some optimizations for list-like constructs.
2. **Simplified subsume relation:** The current implementation only matches exact kind equality and supertypes, missing some valid replacement candidates.
3. **No parallel execution:** The jobs parameter exists but is not implemented. Parallel interest-iness testing would provide near-linear speedup.

5.4.2 5.1.2 Performance Limitations

1. **Full reparse:** Each candidate requires reparsing the entire source. tree-sitter’s incremental parsing could reduce this overhead.
2. **No early termination on large subtrees:** If the entire program is interesting, the reducer tries to delete the root’s children one by one.

6 6. Discussion

6.1 6.1 Design Trade-offs

6.1.1 6.1.1 Accessibility vs. Performance

The dual implementation strategy (Rust primary, Python reference) trades some maintenance overhead for broader accessibility. Python developers can use the reference implementation directly, while performance-sensitive users can use the Rust binary.

6.1.2 6.1.2 Simplicity vs. Completeness

unfluff implements a simplified version of Perses, omitting PNF normalization and the full subsume relation. This reduces implementation complexity but misses some optimizations. The trade-off is justified by the significant reduction in code complexity and the ability to support more languages via tree-sitter.

6.1.3 6.1.3 Generality vs. Specialization

Unlike C-Reduce (which is highly specialized for C/C++), unfluff provides general-purpose reduction across seven languages. This trades some transformation depth for breadth of support.

6.2 6.2 Open Challenges

6.2.1 6.2.1 Global Minimality

Neither Perses nor unfluff achieves global minimality. Vulcan and T-Rec make progress on this front, but the problem remains open. A unified framework combining tree-level and token-level reduction with escape transforms could make further progress.

6.2.2 6.2.2 Incremental Reduction

Current tools reparse the entire source after each transform. Leveraging tree-sitter’s incremental parsing could reduce per-candidate cost by 10–100x for small changes.

6.2.3 6.2.3 Machine-Guided Reduction

Using a lightweight model to predict which candidates are most likely to be accepted could significantly reduce the number of test invocations. This is particularly promising for large programs where the candidate space is vast.

6.2.4 6.2.4 Cross-Language Reduction

For polyglot inputs (e.g., a Python file that embeds C via a build system), reducing across language boundaries simultaneously remains challenging.

6.3 6.3 Broader Impact

Program reduction has a positive impact on software quality:

- **Faster bug triage:** Reduced test cases are easier for developers to understand and fix.
- **Better fuzzing:** Reduced inputs help security researchers understand vulnerabilities.
- **Improved testing:** Reduced test cases are faster to execute and easier to maintain.

However, program reduction could theoretically be misused to:

- **Obfuscate vulnerabilities:** Reducing a trigger to a minimal form could make it harder to understand the attack vector.
- **Bypass security measures:** Reduced test cases could be used to test bypass techniques.

These risks are mitigated by the fact that program reduction requires access to the original program and the interestingness test, which are typically available only to developers and security researchers.

7 7. Conclusion

This paper has surveyed the landscape of program reduction tools and techniques, from the foundational delta debugging algorithm through syntax-guided approaches like Perses to token-level methods like T-Rec. We presented unfluff, a high-performance Rust implementation of the Perses algorithm with multi-language support via tree-sitter.

Key contributions include:

1. **A comprehensive survey** of program reduction tools and techniques.
2. **The unfluff system** with 3–13x speedups over Python and support for seven languages.
3. **Novel features** including pytest integration, error-tolerant reduction, and automatic mode.
4. **An evaluation** against established reducers with performance benchmarks and reduction quality comparisons.

Key design decisions—tree-sitter integration, pytest-based testing, error-tolerant handling, and token-level reduction—make unfluff practical for real-world debugging workflows.

The most impactful improvements, in priority order:

1. **Incremental parsing** (10–100x speedup per candidate)
2. **Full PNF normalization** (complete Perses parity)
3. **Parallel testing** (near-linear speedup with CPU-bound tests)
4. **Token-level expansion** (identifier shortening, string simplification)
5. **Machine-guided reduction** (predictive candidate prioritization)

With these enhancements, unfluff could become the go-to program reducer for multi-language workflows, combining Perses’s theoretical guarantees with tree-sitter’s language coverage and Rust’s performance.

8 Acknowledgments

This work builds on the Perses algorithm by Sun et al. (2018) and the bonsai reference implementation (nnunley 2024). We thank the tree-sitter contributors (Tree-sitter contributors 2024) for providing the parsing infrastructure that makes multi-language support feasible.

References

- DRMacIver. 2023. *Shrink Ray*. <https://github.com/DRMacIver/shrinkray>.
- Misherghi, Ghassan, and Zhendong Su. 2006. “HDD: Hierarchical Delta Debugging.” *Proceedings of the 28th International Conference on Software Engineering*, 106–15.
- nnunley. 2024. *Bonsai: Syntax-Guided Test Case Reducer*. <https://github.com/nnunley/bonsai>.
- Regehr, John, Yang Chen, Pascal Cuoq, Eric Eide, John Ellison, and Xuejun Yang. 2012. “Test-Case Reduction for c Compiler Bugs.” *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation*, 335–46.
- Sun, Chengnian, Yuanbo Li, Qirun Zhang, Tianxiao Gu, and Zhendong Su. 2018. “Perses: Syntax-Guided Program Reduction.” *2018 IEEE/ACM 40th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. <https://doi.org/10.1145/3180155.3180236>.
- Tree-sitter contributors. 2024. *Tree-Sitter: A Parser Generator for Incremental Parsers*. <https://tree-sitter.github.io/>.
- Xu, Zhilei, Yunhui Tian, Mengxiao Zhang, et al. 2025. “T-Rec: Fine-Grained Language-Agnostic Program Reduction Guided by Lexical Syntax.” *ACM Transactions on Software Engineering and Methodology* 34 (2). <https://doi.org/10.1145/3690631>.
- Xu, Zhilei, Yunhui Tian, Mengxiao Zhang, Guqing Zhao, Yu Jiang, and Chengnian Sun. 2023. “Pushing the Limit of 1-Minimality of Language-Agnostic Program Reduction.” *Proceedings of the 2023 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications*. <https://doi.org/10.1145/3586049>.
- Zeller, Andreas, and Ralf Hildebrandt. 2002. “Simplifying and Isolating Failure-Inducing Input.” *IEEE Transactions on Software Engineering* 28: 183–200.